

Aula 6

Inteligência Artificial Aplicada - Machine Learning

Prof. Antonio Willian Sousa

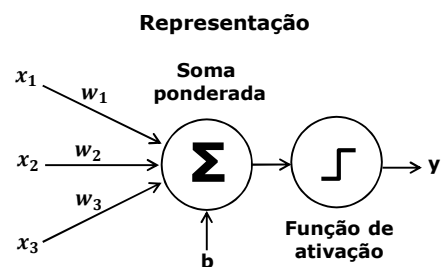
Conversa Inicial

- O Perceptron
- Redes neurais artificiais
- Redes neurais convolucionais
- Redes neurais sequenciais
- *Frameworks* especializados

O Perceptron

Algoritmo do Perceptron

- O Perceptron é um algoritmo
- Possui quatro partes importantes
 1. Valores de entrada (x_1 , x_2 , x_3)
 2. Pesos (w_1 , w_2 , w_3) e *bias* (b)
 3. Soma ponderada
 4. Função de ativação



- ▀ $k = x_1 \cdot w_1 + x_2 \cdot w_2 + x_3 \cdot w_3$ (valores de entrada)
- ▀ $S = k + b$ (soma ponderada)
- ▀ $f(x) = \begin{cases} 0 & \text{se } x < 0 \\ 1 & \text{se } x \geq 0 \end{cases}$ (função de ativação)
- ▀ $y = f(S)$ (resultado final)
- ▀ Calcula o erro: y (valor correto) - y' (predição) = erro

Usando o Perceptron

```
import pandas as pd
from sklearn.linear_model import Perceptron

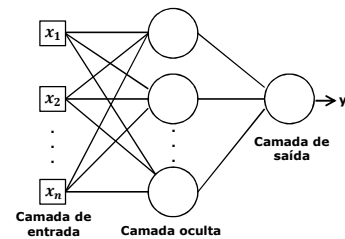
# Separação dos dados
X_treino, X_teste, y_treino, y_teste =
train_test_split(X, y, test_size=0.20)
```

```
# Criação do modelo
percpetron_clf = Perceptron(max_iter=100,
shuffle=True, warm_start=True)

# Treinamento
percpetron_clf.fit(X_treino, y_treino)

# Teste
y_predicao = percpetron_clf.predict(X_teste)
```

Multilayer perceptron – MLP



- ▀ *Feedforward neural network* – FFNN
- ▀ *Artificial neural network* – ANN
- ▀ Diferentes funções de ativação
- ▀ $y = f(x; \theta)$ – descobrir função
- ▀ $f(x) = f_3(f_2(f_1(x)))$

Usando MLP

```
import pandas as pd
from sklearn.linear_model import Perceptron

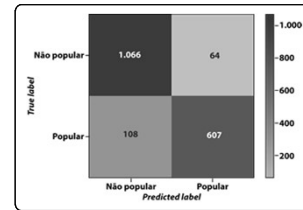
# Separação dos dados
X_treino, X_teste, y_treino, y_teste =
train_test_split(X, y, test_size=0.20)
```

```
# Criação do modelo
mlp_clf = MLPClassifier(warm_start=True,
max_iter=500, hidden_layer_sizes=(100,))

# Treinamento
mlp_clf.fit(X_treino, y_treino)

# Teste
y_predicao = mlp_clf.predict(X_teste)
```

Resultados



■ Acurácia de 91%

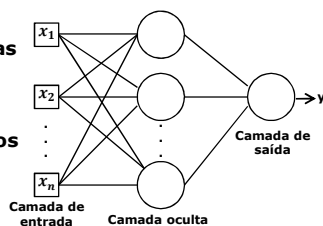
Redes neurais artificiais

Definição do modelo

- **Parâmetros do modelo**
 - São internos ao modelo
 - Exemplo: valor dos pesos dos neurônios
- **Hiperparâmetros do modelo**
 - São externos ao modelo
 - Exemplo: número de neurônios por camada

Hiperparâmetros da estrutura

- Número de camadas ocultas
- Função de ativação
- Inicialização dos pesos
- Taxa de dropout



Hiperparâmetros de treinamento

- Número de *epochs*
- Tamanho do *batch*
- Algoritmo de otimização
- Taxa de aprendizagem

Evitando *overfitting*

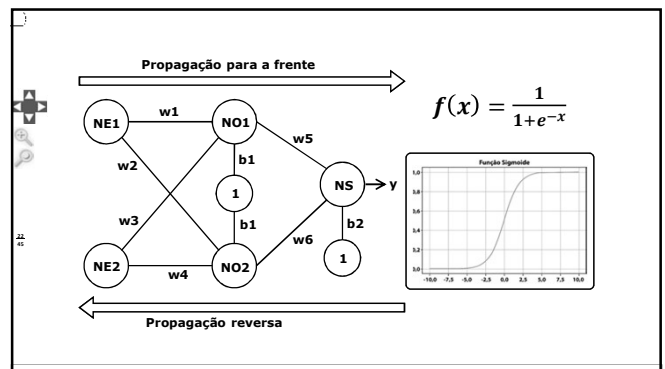
- Retreinar a rede
- Treinar múltiplos modelos
- Monitorar o valor do erro
- Aplicar técnica de regularização

Evitando *underfitting*

- Aumento no número de neurônios
- Aumentar o número de parâmetros
- Mais dados de treino
- Executar *dropout*

Treinamento da rede

- Inicialização dos pesos de todos os neurônios
- Propagação para a frente (*forward*)
- Cálculo do erro
- Propagação reversa (*backpropagation*)
- Atualização dos pesos
- Realiza novas iterações até obter convergência



Ajustes dos hiperparâmetros

- Tentativa e erro
- Grid search*
- Busca aleatória

Busca aleatória

```
mlp_clf = MLPClassifier(warm_start=True,
max_iter=100)
opcoes_parametros = {
    'hidden_layer_sizes':
    [(30,30,30),(20,80),(25,50,5)], 'activation':
    ['tanh', 'relu', 'logistic'],
    'solver': ['sgd', 'adam', 'lbfgs'], 'alpha':
    stats.uniform(0.0001, 0.9),
```

```

'learning_rate': ['constant', 'adaptive']}
random_mlp = RandomizedSearchCV(mlp_clf,
n_iter = 100,

param_distributions = opcoes_parametros)
random_mlp.fit(X_treino_std, y_treino)

```

Grid search

```

grid_mlp = MLPClassifier(warm_start=True,
max_iter=100)
opcoes_parametros = {
'hidden_layer_sizes':
[(30,30,30),(20,80),(25,50,5)],
'activation': ['tanh', 'relu', 'logistic'],

```

```

'solver': ['sgd', 'adam', 'lbfgs'],
'alpha': [0.0001, 0.09],
'learning_rate': ['constant','adaptive']}
grid_mlp = GridSearchCV(mlp_clf, param_grid
= opcoes_parametros)
random_mlp.fit(X_treino_std, y_treino)

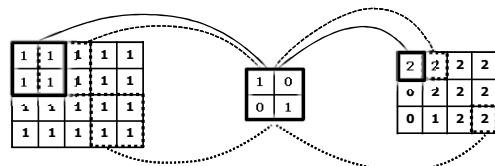
```

Redes neurais convolucionais

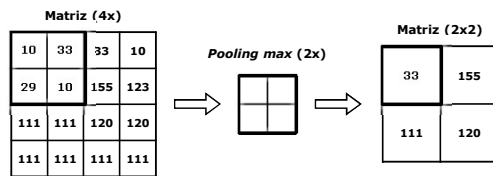
Convolutional neural networks – CNN

- Uma arquitetura de rede neural
- Trabalho com imagens
- Transforma os dados de entrada
- Convoluções e reduções de dimensionalidade

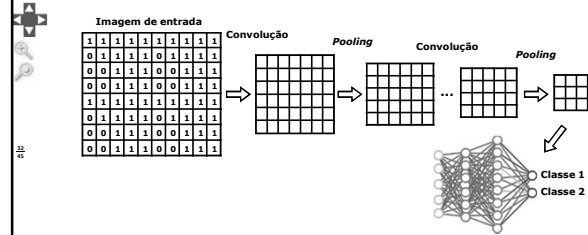
Convolução



Redução de dimensionalidade – *pooling*



Arquitetura



Problemas resolvidos com CNN

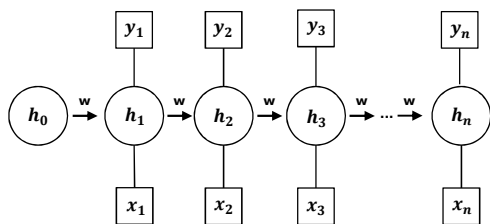
- Classificação de imagens
- Segmentação de objetos
- Detecção de faces
- Reconhecimento facial
- Diversos outros

Redes neurais sequenciais

Recurrent neural networks – RNN

- Redes neurais recebem entradas de tamanho fixo
- Cada instância do conjunto de treino é independente
- Dados como sequências de palavras em uma sentença não podem ser processados
- As redes neurais recorrentes podem fazer isso

- Retêm informações sobre instâncias anteriores
- As instâncias não são independentes
- As entradas e saídas influenciam os resultados futuros



Evolução das RNN

- ▀ O treinamento das RNN é difícil
- ▀ Explosão/desaparecimento do gradiente
- ▀ Memória de curto prazo (*short-term memory*)
- ▀ Memória de longo prazo (*long-term memory*)
- ▀ LSTM (*long short-term memory*)

Problemas resolvidos com RNN

- ▀ Modelagem de linguagem
- ▀ Tradução de máquina
- ▀ Reconhecimento de voz
- ▀ Sumarização de texto
- ▀ Vários outros

Frameworks especializados

Frameworks

- ▀ Tensorflow
- ▀ Pytorch
- ▀ CNTK
- ▀ Keras

Criando uma rede neural com Keras

```
import keras
from keras.models import Sequential
from keras.layers import Dense

model = Sequential()
model.add(Dense(30, input_dim=10,
activation='relu'))
model.add(Dense(30, activation='relu'))
model.add(Dense(2, activation='softmax'))
```

Treinamento do modelo

```
import keras
model.compile(loss='categorical_crossentropy',
              optimizer='adam',
              metrics=['accuracy'])
model.fit(X_treino, y_treino,
          epochs=100,
          batch_size=64)
```

```
Epoch 1/100
116/116 [=====] -
0s 1ms/step -
loss: 0.2962 - accuracy: 0.8695 - auc_1: 0.9480
...
Epoch 100/100
116/116 [=====] -
0s 1ms/step -
loss: 0.1837 - accuracy: 0.9176 - auc_1: 0.9797
```

Monitorando o treinamento

